LISPAI-AOS

Tutorial completo

Dialectos · ORGES · Flows · UI-organismos · Studio · Deploy
AlsetOS · ATS-001 · Runtime declarativo

Este documento enseña LISPAI-AOS de extremo a extremo: conceptos, sintaxis S-expresión, organismos de dominio y de UI, contratos LEGO, reglas ARL, pruebas, despliegue multiplataforma y el uso del Alset Studio (incluido el portable Windows).

Índice

- 1. Qué es LISPAI-AOS y para qué sirve
- 2. Arquitectura: del .aos al Runtime
- 3. Anatomía de un flow
- 4. Organismos (ORGE): dominio y UI
- 5. Contratos LEGO: provides, requires y bind
- 6. Estado y reglas ARL (when / then)
- 7. Superficie UI (IR de componentes)
- 8. Emisiones (emit) y reactividad
- 9. Tests y deploy multi-dispositivo
- 10. Ejemplo completo: cafeteria-mostrador
- 11. Alset Studio (visual + .aos)
- 12. CLI y portable Windows
- 13. Buenas prácticas y límites
- 14. Referencia rápida de formas

1. Qué es LISPAI-AOS y para qué sirve

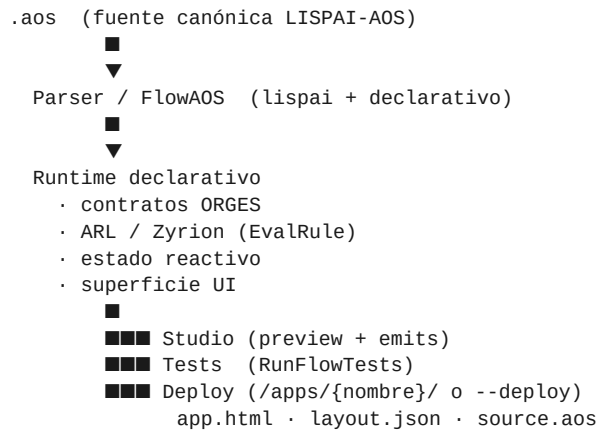
LISPAI-AOS es el dialecto textual de AlsetOS para declarar **organismos (ORGES)**, ensamblarlos como piezas LEGO, probarlos y desplegarlos. La interfaz de usuario también es un organismo (**kind UI**) con un IR de superficie compatible conceptualmente con el UiRuntime del kernel.

No programas pantallas sueltas ni servicios aislados: defines **organismos con identidad, capacidades, contratos, estado y reglas**, y un **flow** los une. El archivo **.aos** es la fuente canónica; el runtime genera artefactos editables (JSON de reglas, superficies, Go) cuando lo necesitas.

Problema que resuelve

- Declarar comportamiento de negocio y UI en el mismo lenguaje.
- Ensamblar módulos con contratos versionados (provides / requires).
- Probar aserciones sobre el estado antes de desplegar.
- Generar previews mobile / tablet / desktop y paquetes de producción por app.

2. Arquitectura: del .aos al Runtime



Relación con el resto de AlsetOS: **declarativo.Runtime** ensambla y ejecuta; **lispai** cubre otras primitivas; el **UiRuntime** del kernel es el destino bare-metal del mismo IR conceptual (column/row/label/button/badge + estado).

3. Anatomía de un flow

Un flow es el contenedor de la aplicación declarativa. Todo vive dentro de **(flow nombre …)**.

```
(flow mi-app
  (version "1.0.0")
  (desc "Descripción legible")

  (orge pieza-a …)
  (orge pieza-b …)
  (orge ui-principal …)

  (assemble pieza-a pieza-b ui-principal)
  (bind pieza-b pieza-a contrato.ejemplo 1.0)
  (surface ui-principal)

  (test nombre-test (eq campo valor))
  (deploy web (devices mobile tablet desktop))
)
```

Orden típico: metadatos → organismos → assemble → binds → surface → tests → deploy. El nombre del flow suele usarse como slug de la app desplegada (a-z, 0-9, guiones).

4. Organismos (ORGE): dominio y UI

4.1 Organismo de dominio

```
(orge stock
  (kind ORGES)
  (desc "Disponibilidad de insumos")
  (cap "stock.consultar" "stock.actualizar")
  (provides "stock.consulta" "1.0")
  (goal "disponibilidad")
  (state (stock.unidades 10) (stock.umbral 3) (stock.alerta false))
  (rule alerta_bajo
    (when (lte stock.unidades 3))
    (then stock.alerta true)))
```

kind ORGES: organismo de negocio. **cap** declara capacidades. **provides** publica un contrato versionado. **goal** es intención legible. **state** inicializa átomos. **rule** define ARL estructurado.

4.2 Organismo UI

```
(orge ui-mostrador
  (kind UI)
  (desc "Superficie declarativa")
  (cap "ui.render")
  (ui
    (column
      (label "Cafeteria · mostrador")
      (badge (bind stock.unidades))
      (row
        (button "+" (emit stock.inc))
        (button "-" (emit stock.dec))
        (button "Vender" (emit venta.registrar))))))
```

kind UI: el organismo *es* la interfaz. El bloque **(ui …)** describe el IR de superficie. Se activa con **(surface ui-mostrador)**.

5. Contratos LEGO: provides, requires y bind

Los organismos no se llaman entre sí por nombre libre: se acoplan por **contratos versionados**.

```
(orge venta
  (kind ORGES)
  (requires "stock.consulta" "1.0")
  …)

(assemble stock venta ui-mostrador)
(bind venta stock stock.consulta 1.0)
```

- **provides** «contrato» «versión» — publica una capacidad contractual.
- **requires** «contrato» «versión» — exige un proveedor compatible.
- **bind** consumidor proveedor contrato versión — unión explícita cuando hay ambigüedad.
- **assemble** lista de OIDs — piezas incluidas en el ensamblaje del flow.

Si varias orges proveen el mismo contrato, **bind** evita resoluciones incorrectas. Trata los contratos como APIs internas versionadas.

6. Estado y reglas ARL (when / then)

El estado es un mapa de átomos (clave → valor). Las reglas ARL se evalúan cuando cambia el estado (p. ej. tras un emit). Zyrion / EvalRule aplican el predicado **when** y, si se cumple, el **then**.

```
(state (stock.unidades 10) (stock.umbral 3) (stock.alerta false))

(rule alerta_bajo
  (when (lte stock.unidades 3))
  (then stock.alerta true))

(rule listo
  (when (gt stock.unidades 0))
  (then venta.habilitada true)
  (indeterminate false))
```

Predicados habituales en when

- **eq** a b — igualdad
- **gt** / **gte** / **lt** / **lte** — comparaciones numéricas

El **then** asigna átomos de estado. Opcionalmente **indeterminate** marca resoluciones parciales según el modelo ARL del runtime.

7. Superficie UI (IR de componentes)

El IR de UI es un árbol de nodos. Componentes soportados en el dialecto ATS-001 del Studio/runtime:

Nodo	Uso
column / row	Contenedores vertical / horizontal
panel	Bloque agrupado
label	Texto estático
badge	Valor (suele ir con bind a estado)
button	Acción: (emit …)
field	Entrada de datos
splash	Pantalla / título de arranque
blob / gradient	Fondos decorativos

drawer / stack	Navegación lateral / apilada
tabs / top-tabs / bottom-tabs	Pestañas
image / file / qr / doclist	Medios y documentos

Enlaces a estado: **(badge (bind stock.unidades))** muestra el valor vivo del átomo. Acciones: **(button "Vender" (emit venta.registrar))**.

8. Emisiones (emit) y reactividad

Al pulsar un botón en preview o producción, el runtime ejecuta el **emit** asociado, muta estado y reevalúa reglas ARL del ensamblaje.

Emisores con efecto en el runtime de Studio

- **stock.inc** / **stock.dec** — incrementan o decrementan stock.unidades.
- **venta.registrar** — registra venta y descuenta stock según reglas del flow.
- **campo=valor** — escribe directamente un átomo de estado.

Puedes ampliar el mapa de emits en el runtime; el contrato del flow debe documentar qué acciones espera cada botón.

9. Tests y deploy multi-dispositivo

9.1 Tests

```
(test stock-inicial (eq stock.unidades 10))
(test sin-alerta (eq stock.alerta false))
```

Tras cargar el flow, el runtime ejecuta las aserciones (RunFlowTests). En Studio: botón **Probar**. En CLI se ejecutan al procesar el flow.

9.2 Deploy

```
(deploy web
  (devices mobile tablet desktop)
  (out ./dist/cafeteria-mostrador))
```

El deploy genera artefactos por dispositivo y, en Studio, una instancia aislada bajo **/apps/{nombre}/** (app.html, previews, source.aos, layout.json). Cada nombre de app tiene su propio Runtime: las instancias no comparten estado.

10. Ejemplo completo: cafeteria-mostrador

Archivo de referencia en el repo: **ejemplos/flows/mostrador.aos**

```
(flow cafeteria-mostrador
  (version "1.0.0")
  (desc "Mostrador: stock, venta, UI-orge, tests y deploy")

  (orge stock
    (kind ORGES)
    (cap "stock.consultar" "stock.actualizar")
    (provides "stock.consulta" "1.0")
    (goal "disponibilidad")
    (state (stock.unidades 10) (stock.umbral 3) (stock.alerta false))
    (rule alerta_bajo
      (when (lte stock.unidades 3))
      (then stock.alerta true)))

  (orge venta
    (kind ORGES)
    (cap "venta.registrar")
    (requires "stock.consulta" "1.0")
    (goal "cobro")
    (state (venta.habilitada false) (venta.ultima null))
    (rule listo
      (when (gt stock.unidades 0))
      (then venta.habilitada true)
      (indeterminate false)))

  (orge ui-mostrador
    (kind UI)
    (cap "ui.render")
    (ui (column
      (label "Cafeteria · mostrador")
      (badge (bind stock.unidades))
      (row
        (button "+" (emit stock.inc))
        (button "-" (emit stock.dec))
        (button "Vender" (emit venta.registrar))))))

  (assemble stock venta ui-mostrador)
  (bind venta stock stock.consulta 1.0)
  (surface ui-mostrador)
  (test stock-inicial (eq stock.unidades 10))
  (deploy web
    (devices mobile tablet desktop)
    (out ./dist/cafeteria-mostrador))
)
```

11. Alset Studio (visual + .aos)

Arranque con Go

```
go run ./cmd/alset-plataforma studio --addr 127.0.0.1:5178
# Abrir http://127.0.0.1:5178/
```

Paneles

- **Toolbox** — piezas ORGE, UI, emits y plantillas arrastrables.
- **Canvas + editor .aos** — fuente canónica editable.
- **Propiedades** — estado vivo y apps desplegadas.
- **Preview** — mobile / tablet / desktop; modo editar layout.
- **Consola** — apply, test, deploy, acciones.

Flujo recomendado en Studio

- 1. Nombre de la app (slug) en la barra superior.
- 2. Arrastra plantilla o escribe el flow en el editor.
- 3. **Aplicar flow** — carga runtime y contratos.
- 4. **Correr preview** — UI viva; prueba botones/emits.
- 5. **Probar** — ejecuta (test …).
- 6. Opcional: editar layout por dispositivo.
- 7. **Desplegar producción** → /apps/{nombre}/.

12. CLI y portable Windows

CLI: probar y generar artefactos

```
# Probar flow
go run ./cmd/alset-plataforma flow ejemplos/flows/mostrador.aos \
  --data /tmp/aos-data

# Generar código editable (JSON / Go / superficies)
go run ./cmd/alset-plataforma flow ejemplos/flows/mostrador.aos \
  --generar /tmp/aos-gen

# Deploy a carpeta
go run ./cmd/alset-plataforma flow ejemplos/flows/mostrador.aos \
  --deploy ./dist/cafeteria
```

Windows sin Go ni npm

En una máquina con Go: **./scripts/build-windows.sh** → **dist/windows/AlsetStudio.exe** + **Iniciar-Alset-Studio.bat**. Copia la carpeta al PC destino, ejecuta el BAT y abre http://127.0.0.1:5178/. Las apps desplegadas quedan en **apps** junto al ejecutable.

Detalle: docs/WINDOWS_STUDIO.md en el repo AlsetOS.

13. Buenas prácticas y límites

Buenas prácticas

- El **.aos** es la fuente de verdad hasta que promociones un artefacto generado.
- Un provides por contrato claro; versiona ("1.0", "1.1"…) al romper compatibilidad.
- Usa bind cuando haya más de un proveedor posible.
- Nombres de app en slug estable para /apps/{nombre}/.
- Versiona los .aos en git junto al producto.
- Documenta los emits que consumen los botones de la UI-orge.

Límites honestos (evolución del dialecto)

- LISPAI-AOS cubre el subconjunto ATS-001 usado en pedidos/cafetería y Studio; no todo el lenguaje aspiracional del paper ATS-001.
- Formas meta (gene / mind / pulse) pueden estar registradas como extensión; verifica soporte en tu build de alset-plataforma.
- El empaquetado USB / MBR-UEFI es capa aparte del dialecto (portable Studio / Core).

14. Referencia rápida de formas (ATS-001)

Forma	Rol
(flow …)	App / ensamblaje contenedor
(orge …)	Organismo (OID, kind, caps, goals)
(provides|requires)	Contratos ORGES versionados
(bind c p contrato ver)	Unión explícita consumidor→proveedor
(state …)	Estado inicial (átomos)
(rule … when/then)	ARL estructurado (Zyrion)
(ui …)	UI-orge / IR de superficie

(assemble …)	Piezas del ensamblaje
(surface …)	Superficie UI activa
(test …)	Verificación de estado
(deploy … devices …)	Empaque multiplataforma
(gene|mind|pulse …)	Meta ATS (extensión)

Documentación en el repo

- docs/LISPAI_AOS.md — visión y flujo de trabajo
- docs/ATS001_LISPAI_AOS.md — tabla de formas alineadas a ATS-001
- docs/WINDOWS_STUDIO.md — portable Windows
- ejemplos/flows/mostrador.aos — flow de referencia

Fin del tutorial LISPAI-AOS · AlsetOS